Lab2 Buffer overflow

Student： *YuChin Chiang u5693885*

Date： 13/11/2025

Environment： school remote Vm

---

1. Background

1) Explain what buffer overflow is

Buffer overflows happen when the program writes more data into a buffer than it had space for. For instance, if a 9-byte array receives a 10-byte value, the extra 1 byte overflows into neighbouring memory. In C, things like strcpy() don't take length into account and the result is that you can write whatever is above your buffer on the stack.

This problem was figured out and exist due to the fact that in the lecture slides:

Heap grows upward and Stack grows downward

Writing “too much” into a buffer might overwrite important stack values→ what's known as stack smashing. This overflow allows the user-controllable input to corrupt other parts of memory that hold critical information, such as saved frame pointers, function parameters, and most dangerously the saved return address.

---

2) Why Buffer Overflows Are Dangerous

As described on the slides, both data and code are just bytes in memory. As a matter of fact in CPUland, what defines if a given region of memory should be seen as code or data is strictly the context of its use.

This means: Data from the attacker can be stored in memory

Assuming the attacker can hijack control29, 31.,+DATA before the time each 20 second scaling window is set into DATA. This is the cornerstone of traditional code-injection attacks. The talk had also touched the historical aspect by bringing in the Morris Worm (1988), which propagated uncontrollably using buffer clobbers, causing $10–100M in damage. It provides evidence buffer

overflows are not just a dangerous theoretical concept, but have led to actual incidents.

When a buffer overflow occurs, writing past the local buffer overwrites:

1. Saved frame pointer
2. Saved return address

Overwriting the return address is the key:

when the function returns, execution jumps to the attacker-provided address.

The ppt emphasizes two exploitation tasks:

- **Task A:** find the offset from the buffer start to the return address
- **Task B:** find an address containing malicious shellcode

Once both are identified, the attacker constructs a malicious input ("badfile") whose tail overwrites the return address with a pointer to the shellcode.

---

3) Techniques Used in Real Attacks

The slides highlight two major techniques:

(1) NOP Sled

A large region of `0x90` (NOP instruction) is placed before the shellcode.

Benefits:

- The attacker doesn't need the exact jump address
- Landing anywhere on the sled → CPU "slides" into the shellcode

(2) Return Address Spraying

If the attacker only knows the rough position of the buffer, but not its exact offset, 4.3 Attack Cause The attack is caused by a statement executing custom SQL code without user input sanitation or whitelisting. They can forge the same guessed return address in multiple instances. This raises the chance that one or more sprayed addresses will litterally overwrite the actual return address on the stack. Both approaches increase robustness on non-ASLR systems drastically.

---

4) Countermeasures (From lecture)

(1) Developer Approaches

- Use the safer functions (strncpy, strncat)
- Perform input validation
- Rely on safe libraries for the bounds checking and so forth

(2) OS Approaches – ASLR

- Attacks, at the time this work was published
- Memory layout randomization
- Makes buffer addresses unpredictable
- More difficult to determine where shellcode resides

(3) Compiler Approaches – StackGuard

- Interposes a canary value between buffer and return address
- If the overflow damages the canary → ABORT() program

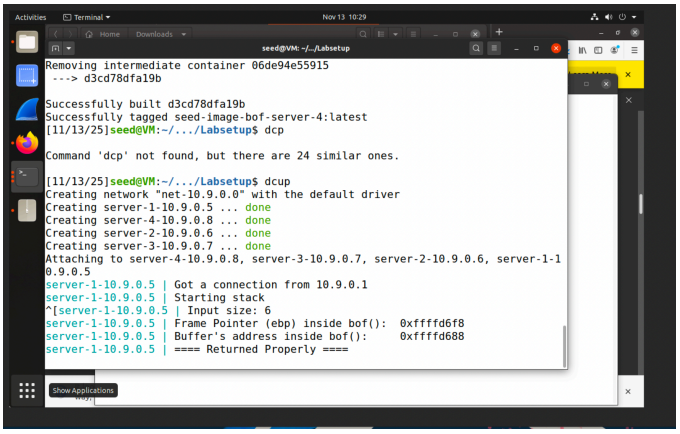
(4) Hardware Approaches – NX Bit

- Marks stack as non-executable
- Protects your CPU from executing injected code
- But can be overcome with return-to-libc

Level 1 Attack

1)Screenshots and Output Explanation

When connecting to server **10.9.0.5:9090**, the server prints



---

2) How I Calculated ret and offset

Buffer Address inside bof() is `0xffffd688`

Shellcode is placed at end of 517-byte buffer that is `content[517 - len(shellcode) : ] = shellcode`

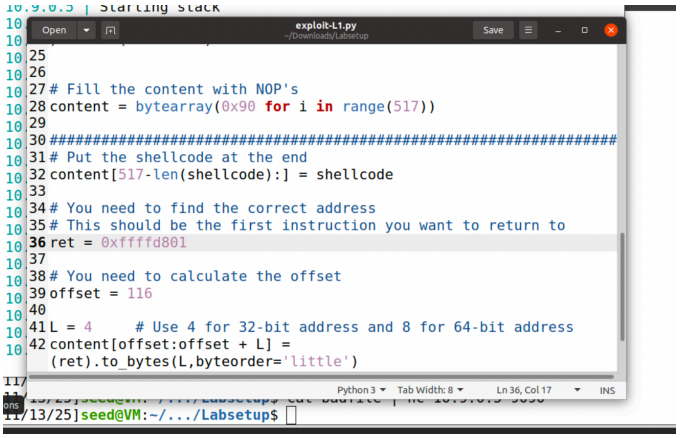
Assume shellcode length ≈ 140 bytes that `517 - 140 = 377`

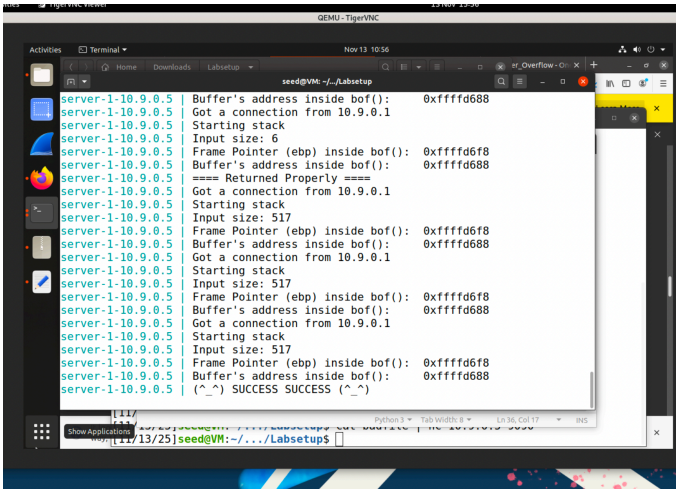
Ret to point inside the NOP sled / shellcode region:

```
ret = buffer + 377
    = 0xffffd688 + 0x179
    = 0xffffd801
```

Offset = distance from buffer start → saved return address that `5buffer size (112 bytes)` + `save EBP(4)` = `116`

Result





Level 2 Attack

1) Screenshots and Output Explanation

When connecting to server **10.9.0.9:9090**, the server prints

```
Starting server-3-10.9.0.7 ... done
Attaching to server-3-10.9.0.7, server-4-10.9.0.8, server-2-10.9.0.6,
0.9.0.5
server-2-10.9.0.6 | Got a connection from 10.9.0.1
server-2-10.9.0.6 | Starting stack
server-2-10.9.0.6 | Input size: 6
server-2-10.9.0.6 | Buffer's address inside bof():     0xffffcf88
server-2-10.9.0.6 | ==== Returned Properly ====
server-2-10.9.0.6 | Got a connection from 10.9.0.1
server-2-10.9.0.6 | Starting stack
server-2-10.9.0.6 | Input size: 517
server-2-10.9.0.6 | Buffer's address inside bof():     0xffffcf88
```

---

2) How I Chose ret and S

Known:

```
buffer = 0xffffcf88
buffer_size ∈ [100, 300]
total input = 517 bytes
shellcode placed at end of buffer
```

Ret calculation

Assume shellcode length ≈ 140 bytes that `517 - 140 = 377`

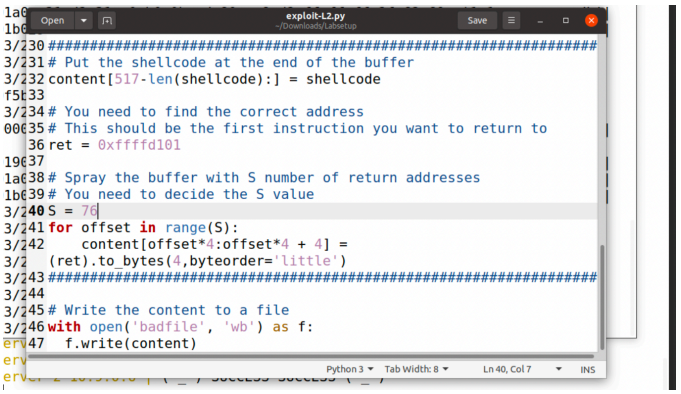
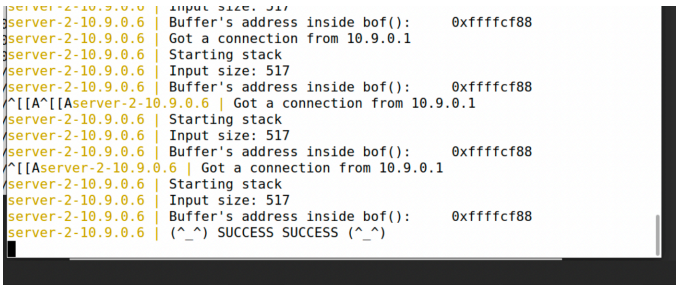
Return target:

```
ret = buffer + 377
    = 0xffffcf88 + 0x179
    = 0xffffd101
```

S Ret calculation

Ensure overwriting the saved return address without knowing buffer size.

Worst-case distance

```
S = (buffer max size + saved ebp + return address) / 4
  = (300 + 4 + 4) / 4
  = 76
```

Result